

Virtual Mouse Using Hand Gestures

A Mini Project Report

By C. Jeremy David

Abstract

In today's world we see lots of development in the field of technology, much of it now combined with Artificial Intelligence. This project applies a small part of AI to a familiar idea: using hand gestures, a way humans have communicated for generations, as a way of communicating with machines instead.

A contactless mouse reduces dependence on physical hardware and, in contexts such as the COVID-19 pandemic, helps avoid the need to touch shared devices. This project presents finger-movement gesture detection on a computer using a standard webcam, letting the whole system be controlled by finger movement alone. It proposes a way to control the cursor position with bare hands, with no electronic device required, while operations like clicking, dragging, and scrolling are performed through different hand gestures. Using finger-detection methods for instant camera access and a user-friendly interface makes the system easy to access. It implements a motion-tracking mouse, a signature input device, and an application selector, reducing the use of a physical mouse and human effort. It is also expressive enough that it could be understood by deaf or mute users. This report presents a real-time hand gesture system.

Introduction

Overview

A virtual mouse is software that allows users to give mouse inputs to a system without using an actual mouse. It can be operated with an ordinary web camera, and can work alongside other input devices such as an actual mouse or a keyboard. In this system, the hand movements of a user are mapped into mouse inputs: a webcam continuously captures images, and image-processing techniques translate hand position into cursor behaviour.

Most laptops today are equipped with webcams, already used for security applications such as face recognition. Harnessing a webcam for vision-based cursor control would effectively eliminate the need for a mouse or mouse pad. The same idea extends to other HCI applications such as sign-language recognition or motion control, following advances already seen in gaming technologies like Microsoft Kinect and Nintendo Wii. Hand-gesture-based HCI is intuitive and effective for one-to-one interaction with computers, providing a Natural User Interface (NUI). Traditional hardware mice have real limitations: they need a flat surface and a certain amount of desk space, and cable length restricts remote interaction — problems a vision-based approach avoids.

Objectives

The main objective of the Virtual Mouse system is to control mouse cursor functions using hand gestures instead of a physical mouse. After starting the tracking application, the user's finger acts as the mouse, and cursor movement follows the direction of the finger.

- Build a program that is a small part of AI-driven interaction.
- Design it to operate using only a standard webcam.
- Support left-click and right-click through gestures.
- Support drag-and-drop through gestures.
- Keep the program's response as smooth as possible.

Motivation

- Vision-based interfaces address the unnaturalness of conventional input devices like mice and joysticks in environments requiring intuitive interaction and teleoperation.
- The virtual mouse is a step toward technology that can be controlled and interacted with remotely, without peripheral devices — offering both convenience and cost savings.
- A physical mouse requires a surface to operate and suffers from cable-length limitations, both of which a camera-based system avoids.
- The motivation is to create an object-tracking application that develops a virtual human-computer interaction (HCI) device.

Project Overview

The application combines AI with computer vision so the camera can recognize each finger's movement and respond accordingly. It uses MediaPipe for hand tracking and the OpenCV Python library to capture and process live video. The goal is to develop the movement of the mouse without any physical contact or use of a mouse device.

Existing virtual mouse systems typically support only simple operations — pointer movement, left click, right click, and drag — using static hand recognition, i.e. recognizing the shape made by the hand and mapping it to a fixed action. This is limited to a few defined actions and can cause confusion. This project extends that with a broader, real-time gesture set.

Analysis and Design

Functional Requirements

The application lets a user access their system through a motion-tracker application. Calculations are all related to motion-detection operations, and data processing uses Python libraries including OpenCV, PyAutoGUI, and MediaPipe.

MediaPipe is an open-source, cross-platform framework from Google, built on time-series data and applicable to audio and video. It is based on three fundamental parts: performance evaluation, a framework for retrieving sensor data, and a collection of reusable components called calculators. A pipeline is a graph of calculators connected by data streams, and developers can replace or define custom calculators anywhere in the graph.

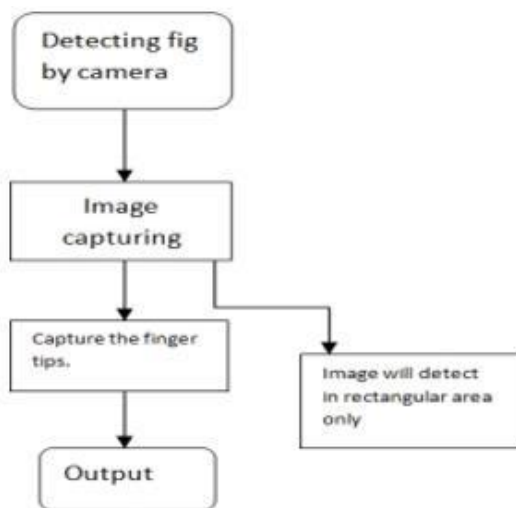


Fig. MediaPipe hand tip recognition module

OpenCV is a computer-vision library containing image-processing algorithms for object detection, used here for image and video processing, and analysis such as face and object detection.

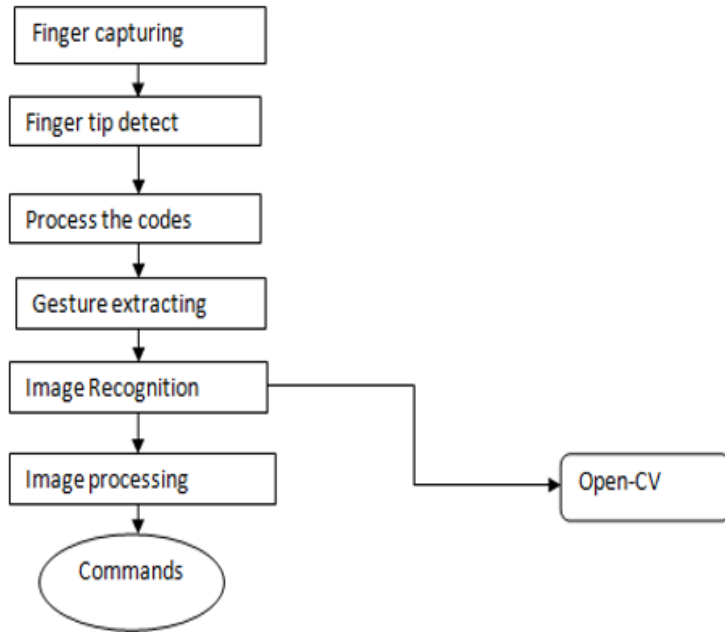


Fig. Hand gesture flowchart

PyAutoGUI lets Python scripts control the mouse and keyboard to automate interactions with other applications; it works on Windows, macOS, and Linux, and runs on Python 2 and 3.

Interface Requirements

The application communicates between the UI and the Python code using Tkinter, the standard GUI library for Python. Tkinter provides a fast, object-oriented interface to the Tk GUI toolkit and ships with standard Linux, Windows, and macOS installs of Python.

Non-Functional Requirements

- Security
- Reliability
- Portability
- Reusability
- Maintainability

Architecture

A system architecture is the conceptual model that defines the structure, behaviour, and views of a system — organized so that products can be procured and systems developed to implement the overall design.

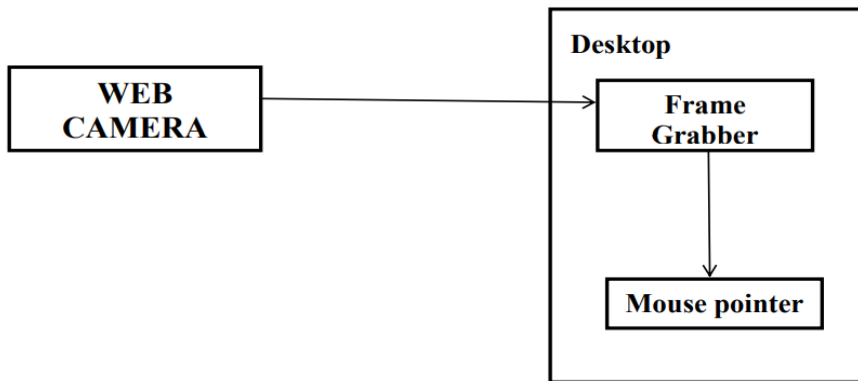


Fig. Architectural representation

Web Camera: a video camera that feeds its image in real time to a computer. It captures light through a small lens using a grid of light detectors that convert the picture into digital format, and transmits frames directly to the computer since it has no built-in memory.

Desktop: most of the processing happens here. It displays the image received from the webcam and provides the interface to the user, comprising a Frame Grabber (which captures individual digital frames from the video stream) and the Mouse Pointer (which moves as the user's hand moves, and whose gestures trigger click events).

Use Case Diagram

A use case diagram summarizes the details of a system and its users, shown as interactions among elements in the system; it specifies the events and how they flow, without describing how they are implemented.

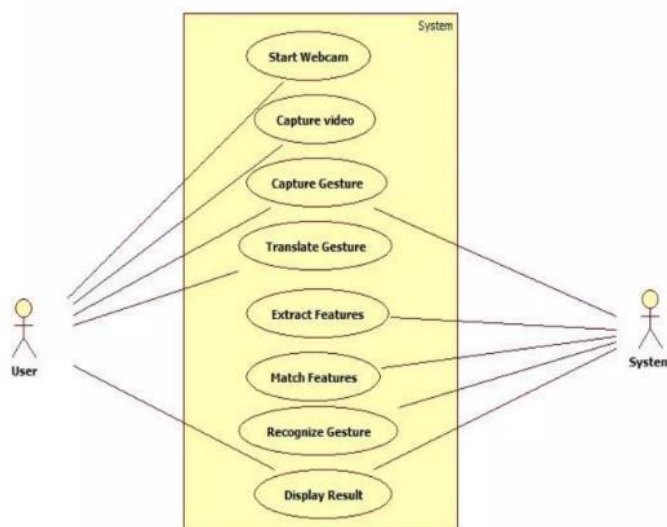


Fig. Virtual mouse use case diagram

Flowchart

The functions and conditions used in the system are explained in the flowchart of the real-time AI virtual mouse system below.

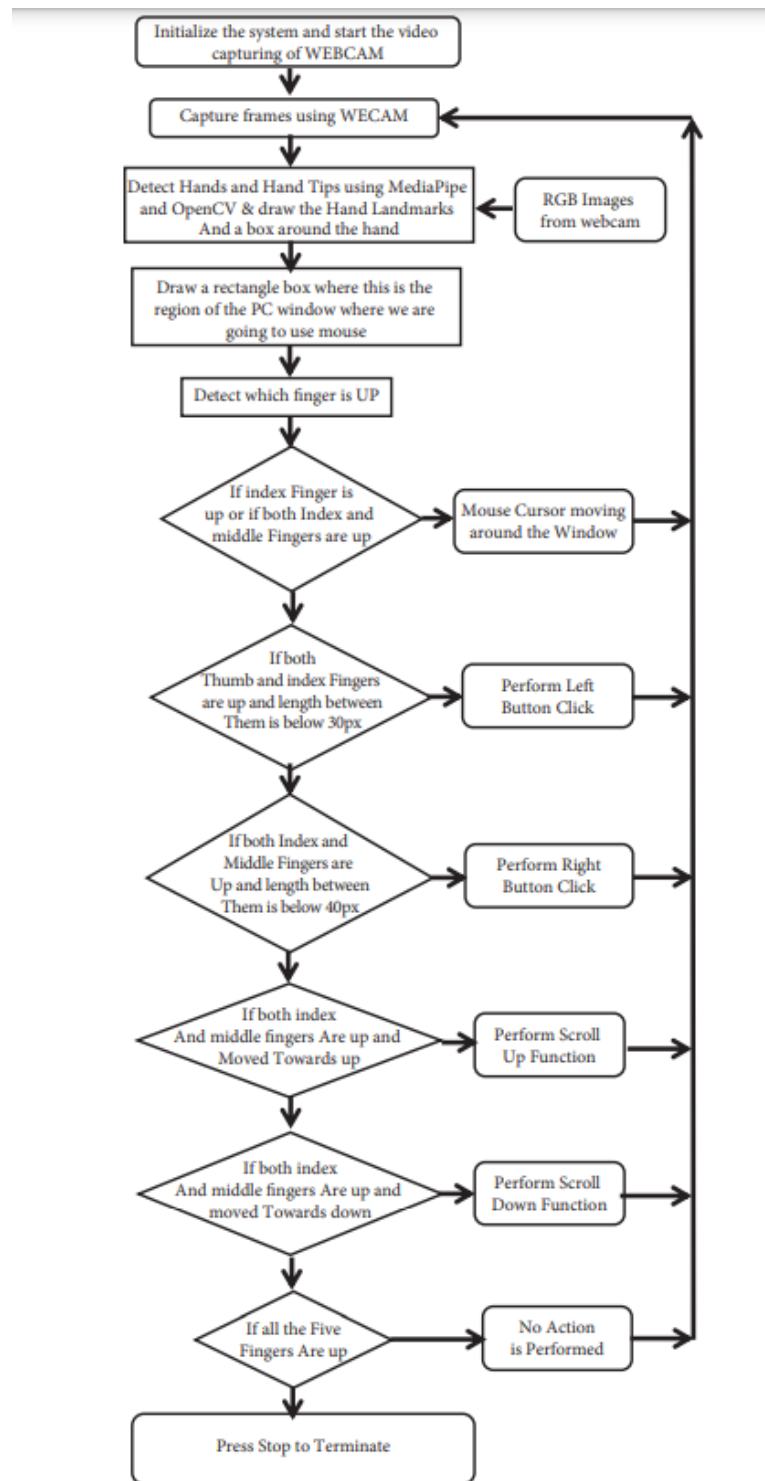


Fig. Virtual mouse flowchart

Implementation

Modules Description

- Planning: reviewing the existing physical-mouse system, identifying problems, and comparing which are most crucial to address.
- Requirement Analysis: gathering and interpreting facts, diagnosing problems, and recommending improvements to the proposed system.
- Designing: prioritizing requirements and preparing the system design, defining the overall architecture and the hardware/software requirements.
- Building: the actual coding implementation, following an agile approach where the developed system is treated as a prototype to be integrated and tested.
- Testing: the prototype undergoes integration and thorough testing by users; feedback drives further modification.

Implementation Details

Camera Used: the system is based on frames captured by a webcam in a laptop or PC. Using the OpenCV video capture object, the webcam starts capturing video and passes frames to the AI virtual system.

Capturing the Video and Processing: each frame is captured until the program terminates, and converted from BGR to RGB colour space so hands can be found frame by frame:

```
def findHands(self, img, draw = True):    imgRGB = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
self.results = self.hands.process(imgRGB)
```

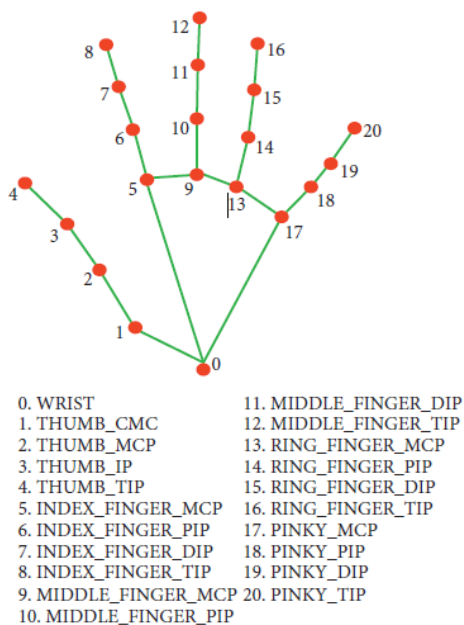


Fig. Hand co-ordinates (21 MediaPipe landmarks)

Rectangular Region for Moving through the Window: a transformational algorithm converts fingertip coordinates from the webcam frame to the full computer screen. Once hands are detected and the active finger identified, a rectangular tracking region is drawn relative to the computer window.

Detecting Which Finger Is Up: the tip ID of each finger, found via MediaPipe, together with its coordinates, determines which finger is raised and which mouse function should run.

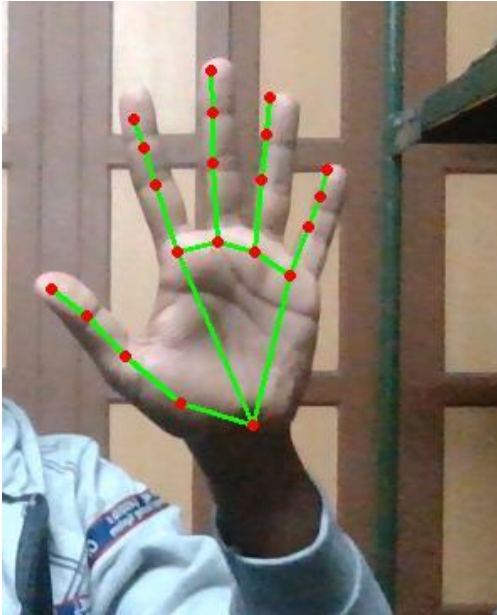


Fig. Detecting hand tip and gestures

Mouse Functions: if the index finger is up (tip ID 1), or both the index (tip ID 1) and middle (tip ID 2) fingers are up, the cursor moves around the screen using PyAutoGUI.



Fig. Controlling cursor with two fingers

Left Click: if both the index finger (tip ID 1) and thumb (tip ID 0) are up and the distance between them is less than 30px, a left mouse click is performed using pynput.

The system uses the cross-platform OpenCV module and implements mouse actions with Python-specific libraries. Real-time video from the webcam is processed, and only the relevant fingertips are extracted; their centres are measured from webcam fingertip moments, and the action taken depends on their relative positions and distance from one another. `cv2.VideoCapture()` captures the live video stream, and captured images are converted to HSV format for processing.

Tools Used

Python: a high-level, general-purpose language emphasizing readable, significantly-indented code. It is dynamically typed and garbage-collected, and supports structured, object-oriented, and functional programming paradigms.

OpenCV: a library of programming functions aimed mainly at real-time computer vision, originally developed by Intel and later supported by Willow Garage and Itseez. It is cross-platform, free under the Apache 2 license, and has included GPU acceleration for real-time operations since 2011.

VS Code: a source-code editor made by Microsoft with the Electron framework, for Windows, Linux, and macOS, supporting debugging, syntax highlighting, code completion, snippets, refactoring, and embedded Git.

Test Results

Cross-comparison testing is difficult since only a limited number of datasets are available for this kind of system. Hand-gesture and fingertip detection were tested under various illumination conditions and at different distances from the webcam. The test was run 10 times by 3 people (90 gestures total, manually labelled), across different light conditions and distances: 10 times in normal light, 5 times in faint light, 5 times at close distance, and 5 times at long distance from the webcam.

Mouse Function Performed	Success	Failure	Accuracy (%)
Mouse movement	100	0	100%
Left button click	98	2	98%
Right button click	99	1	99%
Scroll function	93	7	93%
No action performed	100	0	100%
Overall Result	681	19	97.28%

The system achieved an overall accuracy of about 97%, indicating it performed well. Accuracy is lowest for the scroll function, as its gesture is the hardest for the system to interpret; accuracy for all other gestures is high.

Test Cases

Scenario	Boundary Value	Expected Result	Actual Result	Status
Normal environment	>90%	Gestures recognized easily	Recognized easily and worked properly	Passed
Bright environment	>60%	Should work fine detecting hand movements	Worked very well	Passed
Dark environment	<30%	Should work properly	Did not detect gestures properly	Failed
Far distance (35cm)	>95%	Should work fine	Worked properly	Passed
Farther distance (60cm)	>60%	Some detection issues expected, but should work	Functions worked properly	Passed

Applications and Conclusion

Applications

- Amid situations like COVID-19, avoids the need to touch shared devices, reducing risk of spreading illness.
- Can be used to draw 2D and 3D images via hand gestures.
- Can control virtual-reality and augmented-reality games without a wired or wireless mouse.
- Can help people with hand-related conditions operate mouse functions on a computer.
- Applicable in robotics for controlling robots via HCI.
- Applicable in design and architecture for virtual prototyping.

Future Scope

There is a lot of room to develop these techniques further. A colour-detection model could identify a particular colour from an image, and mouse movement could be refined to act more like a real mouse. Future work includes implementing additional gestures for more functions, and extending the system — which currently uses only one hand — to support both hands for different gesture combinations.

- Smart Movement: an adaptive zoom in/out to extend the current ~25cm tracking radius, automatically adjusting focus based on distance from the webcam.
- Better Accuracy & Performance: response time depends on hardware (processor speed, RAM, webcam quality), so performance will improve on stronger machines and better-lit conditions.
- Mobile Application: porting the concept to Android, replacing touchscreen input with hand gestures.

Conclusion

The main objective of the AI virtual mouse system is to control mouse cursor functions using hand gestures instead of a physical mouse. It is achieved using a webcam or built-in camera that detects hand gestures and fingertips, processing frames to perform mouse functions. Because accuracy and efficiency are essential to making the program as useful as a physical mouse, several techniques were needed to get there. With this application, there is no need for any physical mouse — every movement a physical mouse would make is instead performed through this motion-tracking virtual mouse.

References

- 1) OpenCV Website – www.opencv.org
- 2) MSDN Microsoft Developer Network – www.msdn.microsoft.com
- 3) Code Project – www.codeproject.com/Articles/498193/Mouse-Control-via-Webcam
- 4) Aniket Tatipamula's Blog – <http://anikettatipamula.blogspot.in/2012/02/hand-gesture-using-opencv.html>
- 5) Microsoft Research Paper – <http://research.microsoft.com/en-us/um/people/awf/bmvc02/project.pdf>
- 6) Google, MediaPipe – <https://ai.googleblog.com/2019/08/on-device-real-time-hand-tracking-with.html>
- 7) L. Thomas, "Virtual mouse using hand gesture," International Research Journal of Engineering and Technology (IRJET), vol. 5, no. 4, 2018.
- 8) MediaPipe Hands – <https://google.github.io/mediapipe/solutions/hands>
- 9) AutoPy Documentation – <https://www.autopy.org/documentation/apireference/mouse.html>
- 10) K. H. Shibly, S. Kumar Dey, M. A. Islam, and S. Iftekhar Showrav, "Design and development of hand gesture based virtual mouse," in Proceedings of the 2019 1st International Conference on Advances in Science, Engineering and Robotics Technology (ICASERT), pp. 1–5, Dhaka, Bangladesh, May 2019.